

Host guide: Smart-RUDP review

For the presenter only. Each page shows one slide of the real deck, what to say, and the question it is likely to bring.

Computer Networks | Faculty: Anuradha G | 9 Oct 2026

Plan for the session

Total talking time about 15 minutes, then questions.

#	Slide	Time	#	Slide	Time
1	Title	0:20	13	Network design	0:50
2	The problem	0:40	14	Packet Tracer results	0:40
3	Why we built it	0:50	15	Smart-RUDP design	0:50
4	What we built	0:40	16	The AI agent in our code	1:00
5	Packet header	0:40	17	Smart-RUDP result	0:50
6	Handshake and data flow	0:50	18	Wireshark traces	0:40
7	Go-Back-N	0:50	19	Wireshark real capture	0:50
8	AIMD	0:50	20	RUDP Chat demo	1:00
9	How we tested	0:40	21	More evidence: baselines, SACK, reordering	1:00
10	Benchmark results	0:40	22	Learnings and future work	0:50
11	Problems fixed	0:40	23	Thank you	0:15
12	Literature	0:30	24	Appendix: slow start	-

Rules: say the limits before she asks; answer in one or two sentences; if you do not know, say what you would check.

1. Title

Time on this slide: 0:20

Smart-RUDP

Reliable UDP with learning-based congestion control, in C

Computer Networks | Faculty: Anuradha G

Harshavardhan Mallela 24BAI0274 Rohit Arya 24BCE0819 Shloak Sinha 24BDS0378

SAY

- Good morning ma'am. We are Harshavardhan, Rohit and Shloak.
- Our project is Smart-RUDP: a reliable transport protocol we built in C on top of UDP, plus a controller that learns how fast to send.

IF ASKED Say the three names and the GitHub link once, then move on.

2. The problem

Time on this slide: 0:40

The problem

UDP

Fast and simple, but no guarantees: packets can be lost, duplicated or reordered.

TCP

Reliable, but lives inside the OS kernel. Hard to see, hard to change, hard to learn from.

In 1986 the early Internet suffered congestion collapse: throughput fell from 32 Kbps to 40 bps. TCP congestion control was invented to fix it.

Our question: can we rebuild TCP's core ideas ourselves, on top of UDP?

2 / 23 Smart-RUDP

SAY

- UDP is fast but gives no guarantees: packets can be lost, duplicated or reordered.
- TCP is reliable but sits inside the operating system, so you cannot see or change it.
- In 1986 the early Internet had congestion collapse: throughput fell from 32 Kbps to 40 bps. TCP's congestion control was invented to fix that.
- Our question: can we rebuild TCP's core ideas ourselves, on UDP?

IF ASKED Q: Why not just use TCP? A: TCP cannot be changed or inspected; building our own makes every mechanism visible. QUIC and HTTP/3 do the same in practice.

3. Why we built it

Time on this slide: 0:50

Why we built it

The need

Campus users send data to a server across a routed network. TCP is fixed inside the kernel; plain UDP gives no guarantees. Real systems (QUIC, HTTP/3) solve this with reliability built in user space over UDP.

Our answer

RUDP: a readable, tunable reliable transport over UDP, plus Smart-RUDP, a controller that learns the window from RTT and loss.

Scenario: a student on VLAN 10 uploads a file to the server LAN. The ACL keeps students out of staff VLAN 20. If a core link fails, OSPF reroutes through R2: a longer path and higher RTT that the transport must adapt to.

Validated: protocol tests and Smart-RUDP vs AIMD on emulated paths; network design built and tested in Packet Tracer; traffic analysed in Wireshark.

3 / 23 Smart-RUDP

SAY

- The need: campus users send data to a server across a routed network. TCP is fixed in the kernel; UDP gives no guarantees.
- Our answer: RUDP, a readable and tunable transport, and Smart-RUDP, which learns the window from RTT and loss.
- Scenario: a student on VLAN 10 uploads to the server LAN. The ACL keeps students out of staff VLAN 20. If a core link fails, OSPF reroutes through R2: longer path, higher RTT, and the transport must adapt.

IF ASKED Q: Is this just a lab exercise? A: A lab sends datagrams. We built reliability and congestion control on top, measured them, and used them in an application.

4. What we built

Time on this slide: 0:40

What we built

Handshake

SYN, SYN-ACK, ACK
before any data flows.

Reliability

Sequence numbers,
ACKs, checksums,
timeouts,
retransmission.

Flow control

Go-Back-N sliding
window; its size is the
congestion window.

Congestion control

AIMD: slow start,
avoidance, halve on
loss.

About 670 lines of C across 4 source files. Client + server, built in 8 phases, on github.com/Capsturrrr/rudp

4 / 23 Smart-RUDP

SAY

- Four pieces: handshake, reliability, flow control and congestion control.
- About 670 lines of C across four source files, built in eight phases, on GitHub at [Capsturrrr/rudp](https://github.com/Capsturrrr/rudp).

IF ASKED Q: Who did what? A: Agree one answer as a team before you go in.

5. Packet header

Time on this slide: 0:40

The RUDP packet header: 13 bytes

Field	Size	Purpose
seq	4 B	Packet number
ack	4 B	Next packet expected
flags	1 B	SYN / ACK / FIN / DATA
checksum	2 B	Detects corruption
payload_len	2 B	Data length (max 512)

Fields packed byte by byte, so compiler struct padding never changes the wire format.

Data packet = 53 bytes on the wire. ACK / handshake = 13 bytes.

5 / 23 Smart-RUDP

SAY

- 13-byte header: sequence 4, ack 4, flags 1, checksum 2, payload length 2.
- Fields are packed byte by byte so compiler padding never changes the wire format.
- A data packet is 53 bytes on the wire; ACKs and handshake packets are 13.

IF ASKED Q: How does the checksum work? A: 16-bit one's-complement sum over header and payload, like UDP/TCP. A corrupted packet fails and is recovered by retransmission.

6. Handshake and data flow

Time on this slide: 0:50

Handshake and data flow

1	SYN	Client -> Server	Request a connection
2	SYN-ACK	Client <- Server	Server agrees
3	ACK	Client -> Server	ESTABLISHED, data may flow
4	DATA	Client -> Server	Up to window-size packets in flight
5	ACK	Client <- Server	Cumulative: confirms everything up to N
6	FIN	Client -> Server	Close the connection

Server accepts a new SYN in any state, so a killed client can never leave it stuck.

6 / 23 Smart-RUDP

SAY

- Three-way handshake: SYN, SYN-ACK, ACK. Then data, up to a window of packets in flight.
- ACKs are cumulative: ACK N confirms everything before N. FIN closes the connection.
- The server accepts a new SYN in any state, so a killed client can never leave it stuck. That was a real bug we fixed.

IF ASKED Q: What if the peer disappears? A: Sender keeps retransmitting on timeout. An abort after repeated timeouts is not implemented; it is the fix we would add.

7. Go-Back-N

Time on this slide: 0:50

Reliability: Go-Back-N

- Window = cwnd packets in flight (fixed at 4 in Phase 4, then driven by cwnd)
- Receiver sends cumulative ACKs
- Retransmission timeout: 500 ms
- Fast retransmit after 3 duplicate ACKs, no waiting for the timer
- Checksum failures are dropped, then recovered by retransmission

Proof

In a netem 20% loss capture we saw 12 timeouts spaced about 500 ms apart.

Result

2,442-byte message reassembled correctly.

7 / 23 Smart-RUDP

SAY

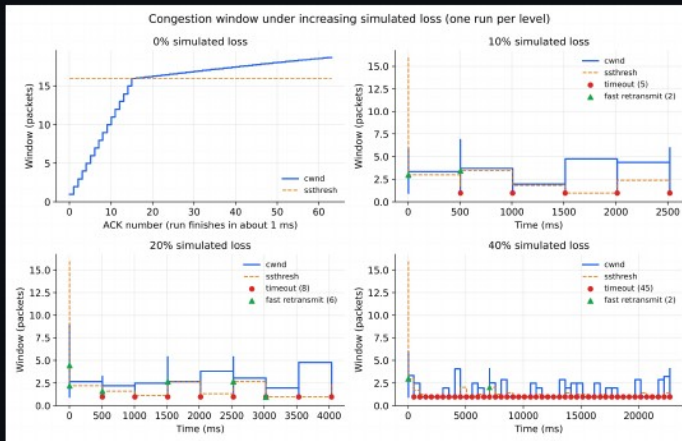
- The window is the number of packets in flight; it started fixed at 4 and is now driven by the congestion window.
- Timeout is 500 ms; fast retransmit triggers after 3 duplicate ACKs, no waiting for the timer.
- Proof: in a 20% loss netem capture we saw 12 timeouts about 500 ms apart. Result: the 2,442-byte message reassembled correctly.

IF ASKED Q: Why Go-Back-N? A: Simpler to build and explain. It wastes bandwidth under loss; Selective Repeat with SACK is our stated future work.

8. AIMD

Time on this slide: 0:50

Congestion control: AIMD



Fresh runs at 0%, 10%, 20% and 40% simulated loss. Red = timeout, green = fast retransmit.

8 / 23 Smart-RUDP

SAY

- Slow start: window grows by one per ACK until the threshold of 16. Then avoidance: plus $1/\text{window}$ per ACK.
- Three duplicate ACKs: window halves. A timeout: window resets to 1 and the threshold halves. Cap is 32.
- Timeout: cwnd resets to 1, ssthresh halves
- Capped at 32
- The graph shows fresh runs at 0, 10, 20 and 40 percent loss. Red marks are timeouts, green are fast retransmits.

IF ASKED Q: Why cap 32 and ssthresh 16? A: The cap stops a lossless run growing without bound; 16 is where doubling switches to linear growth.

9. How we tested

Time on this slide: 0:40

How we tested

Built-in loss simulator

Drops packets in code at a set percentage. Used for the benchmark.

tc / netem

Linux kernel injects real loss on loopback. Used to validate behaviour.

Wireshark / tshark

Captured in WSL, opened in Windows Wireshark. Confirmed handshake, ACKs, retransmissions.

Wireshark could not see WSL loopback from Windows, so we captured with tshark inside WSL and copied the file across.

9 / 23 Smart-RUDP

SAY

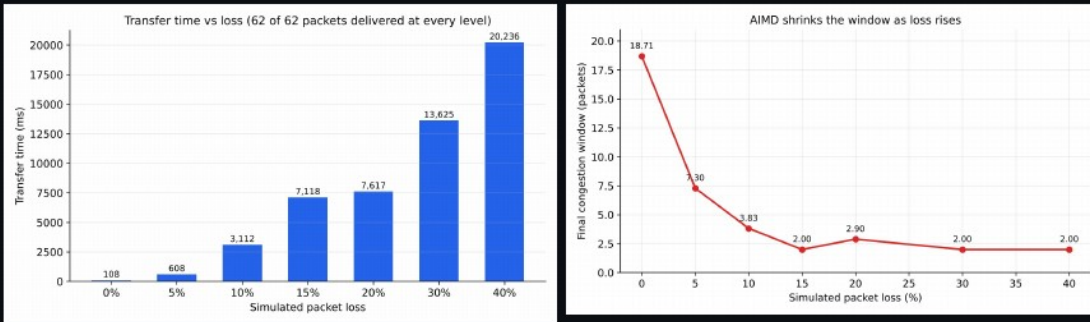
- Three tools: a loss simulator inside the code (used for the benchmark), Linux tc/netem injecting real loss on loopback, and Wireshark/tshark.
- Windows Wireshark could not see WSL loopback, so we captured with tshark inside WSL and copied the file across.

IF ASKED Q: Is the loss real? A: Benchmark and chat loss is simulated in the program. We also captured a real loopback run with tshark and netem loss.

10. Benchmark results

Time on this slide: 0:40

Benchmark results



62 of 62 packets delivered at every loss level, from 0% to 40%.

10 / 23 Smart-RUDP

SAY

- 62 of 62 packets were delivered at every loss level from 0 to 40 percent.
- Higher loss costs time and retransmissions, never correctness.

IF ASKED Q: Single runs? A: Yes, the original benchmark ran each level once. Smart-RUDP evaluation uses 6 runs per scenario.

11. Problems fixed

Time on this slide: 0:40

Problems we hit and fixed

Problem	Fix
Server stuck after killed client	Accept SYN in any state (was a 15.5 s hang)
Leftover netem rule	Check tc qdisc show dev lo, delete before each test
Windows Wireshark saw nothing	Capture with tshark inside WSL, then copy
Shell portability in benchmark	Replaced date/grep tricks with awk and sed

11 / 23 Smart-RUDP

SAY

- Server stuck after a killed client: it hung 15.5 s; fix was to accept SYN in any state.
- A leftover netem rule distorted results; we now check tc qdisc before each test.
- Windows Wireshark saw nothing: capture inside WSL.
- Benchmark script portability: replaced date and grep tricks with awk and sed.

IF ASKED Say these quickly: it shows real debugging, which is what separates a project from a lab.

12. Literature

Time on this slide: 0:30

Literature behind the design

- Jacobson & Karels (1988): congestion avoidance and control, the origin of AIMD
- Chiu & Jain (1989): why additive increase, multiplicative decrease converges fairly
- RFC 5681 (2009): TCP congestion control standard
- Ha, Rhee, Xu (2008): CUBIC, our future-work direction
- Langley et al. (2017) and RFC 9000: QUIC, reliable transport over UDP in production
- Midhula & Arun Raj Kumar (2024) and Cao et al. (2023, DRLFcc): deep reinforcement learning for congestion control, the idea behind Smart-RUDP

Key researcher: Van Jacobson

12 / 23 Smart-RUDP

SAY

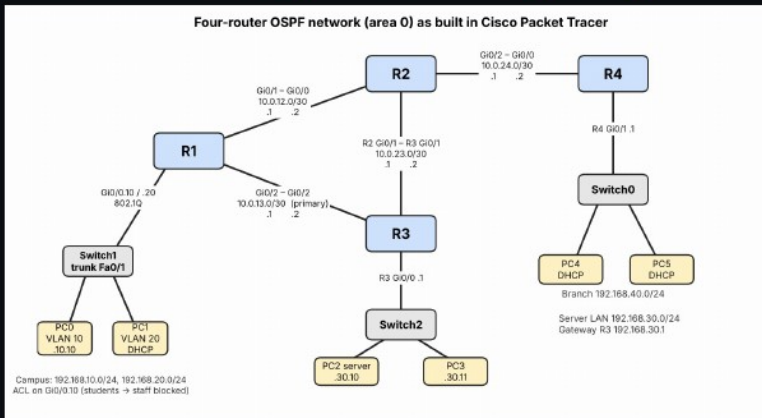
- Jacobson and Karels 1988 is the origin of AIMD; Chiu and Jain 1989 explains why it converges fairly.
- RFC 5681 is the standard; CUBIC is future work; QUIC and RFC 9000 are reliable transport over UDP in production.
- The AI part comes from Midhula and Arun Raj Kumar (2024) and Cao et al. (2023, DRLFcc): deep reinforcement learning for congestion control. Smart-RUDP applies the same idea with a smaller tabular Q-learning agent.
- Key researcher: Van Jacobson.

IF ASKED Q: Why AIMD? A: Chiu and Jain show it converges to a fair share. Q: Are those papers deep RL and yours is not? A: Yes; we used a lookup table, which is simpler and inspectable.

13. Network design

Time on this slide: 0:50

Network design: OSPF topology in Packet Tracer



13 / 23 Smart-RUDP

SAY

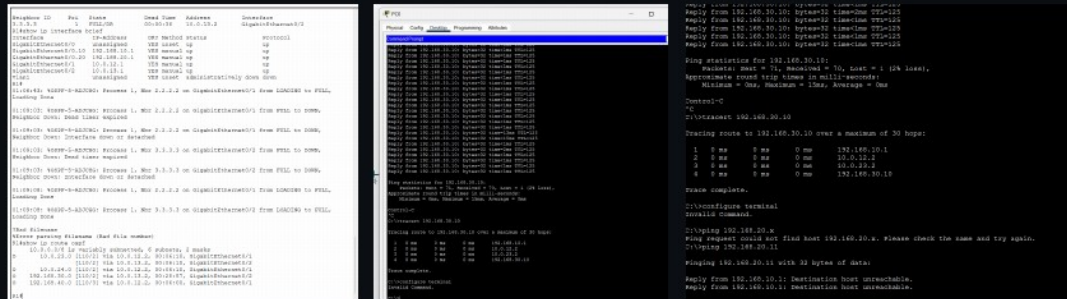
- Four routers, OSPF area 0: R1, R2, R3 form a triangle and R4 is a branch.
- Two VLANs with router-on-a-stick, DHCP on R1 and R4, and an access list blocking student to staff.
- Routes: server LAN metric 2 via R3, branch metric 3 via R2
- Failover: shut R1 Gi0/2, 70/71 pings, reroute via R2 (metric 3)

IF ASKED Q: Does RUDP run inside Packet Tracer? A: No, it cannot run C. Packet Tracer validates the network design; protocol is validated on one machine plus Wireshark.

14. Packet Tracer results

Time on this slide: 0:40

Packet Tracer results: routes, failover, ACL



OSPF routes on R1

```
Link failure: 70/71 pings,
reroute
```

ACL: student blocked, staff allowed

- Server LAN metric 2 via R3, branch LAN metric 3 via R2
- Shutting R1 Gi0/2: traffic reroutes via R2, only 1 of 71 pings lost
- Student PC gets "destination unreachable" to staff; server still reachable (TTL 126)

SAY

- Left: OSPF routes on R1 with the metrics. Middle: the link-failure test, 70 of 71 pings. Right: the ACL, where the student gets destination unreachable to staff but the server is still reachable (TTL 126).

IF ASKED If asked to show it live: have the .pkt file open and follow Demo_Steps.txt.

15. Smart-RUDP design

Time on this slide: 0:50

Smart-RUDP: a Q-learning window controller

State (80)

RTT ratio x loss fraction x window size, sampled once per RTT.

Action (5)

Multiply cwnd by 0.5, 0.85, 1.0, 1.15 or 1.5.

Reward

Throughput gained, minus delay growth, minus retransmissions.

Same Go-Back-N reliability as the baseline; only the window policy changes. Paths emulated in-program: 5G-like, lossy wireless, satellite-like.

15 / 23 Smart-RUDP

SAY

- Q-learning with 80 states (RTT ratio, loss fraction, window size), sampled once per RTT.
- Five actions: multiply the window by 0.5, 0.85, 1.0, 1.15 or 1.5.
- Reward: throughput gained, minus delay growth, minus retransmissions. Same Go-Back-N reliability; only the window policy changes. Paths are emulated in the program: 5G-like, lossy wireless, satellite-like.

IF ASKED Q: Is it really learning? A: Yes, 40 training episodes; the learning-curve figure shows it. It is small and tabular with limited training.

16. The AI agent in our code

Time on this slide: 1:00

The AI agent in our code

```
src/rl_cc.h (agent, simplified)
// state: 4 x 4 x 5 = 80 buckets
s = (rtt_bucket*4 + loss_bucket)*5 + cwnd_bucket;

// choose: explore with prob. eps, else best
int rl_choose(a, s) {
    if (rand01() < a->eps) return rand() % 5;
    return argmax(a->q[s]); // 5 actions
}

// learn: the Q-learning update
void rl_update(a, s, act, r, s2) {
    target = r + a->gamma * max(a->q[s2]);
    a->q[s][act] += a->alpha * (target - a->q[s][act]);
}
```

```
src/smart_client.c (decision loop, simplified)
// once per RTT, in smart_client.c
ratio = rtt / base_rtt; lossf = losses / sent;

reward = sqrt(throughput_norm)
        - 0.5 * max(0, ratio - 1) // delay growth
        - 4.0 * lossf;           // loss

rl_update(&ag, prev_s, prev_a, reward, s2);
act = rl_choose(&ag, s2);
cwnd *= {0.5, 0.85, 1.0, 1.15, 1.5}[act];
```

Tabular Q-learning: alpha 0.15, gamma 0.9. Chat version: web/cc.py, trained in web/train.py (results/q_chat.txt). Deep version: web/dqn.py, a small neural network (5-32-32-5) trained by DQN in web/train_deep.py.

16 / 23 Smart-RUDP

SAY

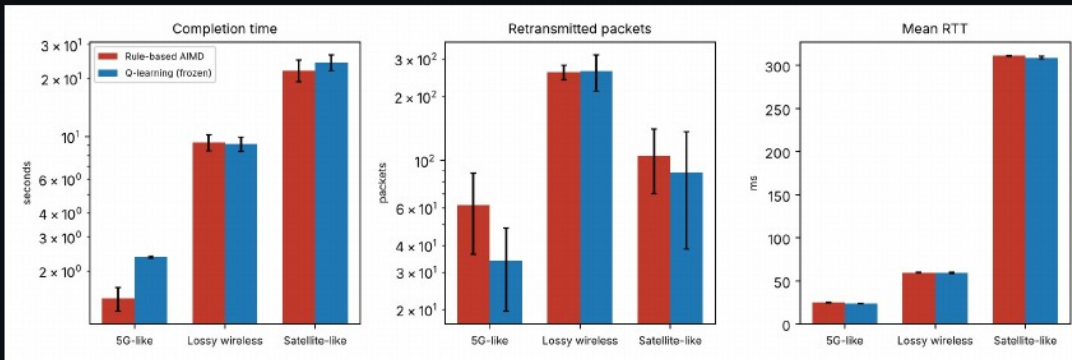
- This is the agent itself. Left: rl_cc.h, 73 lines of C. The state is 80 buckets of RTT ratio, loss and window size.
- choose picks one of five window multipliers, usually the best, sometimes random so it keeps exploring.
- update is the Q-learning rule: move the value of the action we took toward the reward plus the best value of the next state.
- Right: the loop in smart_client.c. Once per RTT it computes the reward, throughput minus delay growth minus loss, updates the table, picks the next action and scales the window.
- The code on the slide is simplified; the real files are in the repo.
- We also built a deep version: web/dqn.py is a small neural network, two hidden layers of 32, trained with DQN in the same simulator. It reads continuous inputs instead of buckets.

IF ASKED Q: Is this deep RL? A: The C agent is tabular. The chat also has a deep agent, a small DQN in numpy (5-32-32-5). Q: Does deep beat the table? A: Modestly: 3.20 s vs 3.73 s mean on 48 unseen paths, faster on 19, slower on 6. Q: Any problem? A: Training is seed-sensitive: 1 of 4 seeds failed, so we select on validation paths. It is simulated and not in the C code. Q: Where is it trained? A: web/train.py for the chat, the smart client trains online for the main experiments; table in results/q_chat.txt.

17. Smart-RUDP result

Time on this slide: 0:50

Smart-RUDP vs AIMD: honest result



- 6 evaluation runs per scenario after 40 training episodes
- Not faster: +63% time on 5G-like, +10% on satellite, tie on lossy
- Fewer retransmissions on 5G-like and satellite, but within run-to-run noise
- Earlier training rounds were much worse; reported run is the final configuration

17 / 23 Smart-RUDP

SAY

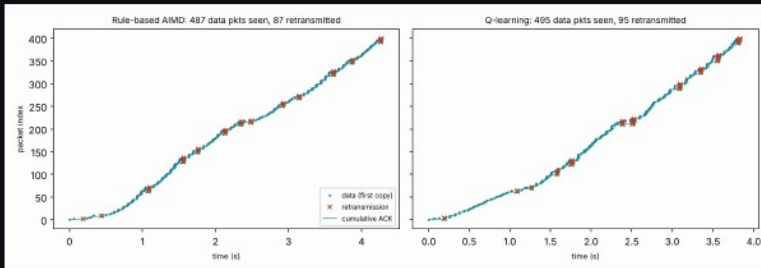
- Be honest: it did not beat AIMD. Plus 63 percent time on 5G-like, plus 10 percent on satellite, a tie on the lossy path.
- It used somewhat fewer retransmissions on two paths, within run-to-run noise.
- Earlier training rounds were much worse; this is the final configuration. The finding is that a simple learner does not beat a tuned rule.
- In the chat, retrained for bursty random-loss traffic, it does beat AIMD on most simulated paths; mention that on slide 19.

IF ASKED Q: Then what was the point? A: The experiment. We tested whether a learned window helps and report the answer, including the negative result.

18. Wireshark traces

Time on this slide: 0:40

Wireshark: RUDP dissector and traces



- rudp.lua decodes seq, ack, flags, checksum, length
- Filters: `smartrudp.flags.data == 1`, `smartrudp.seq == N`
- 400 packets, lossy path: 87 (AIMD) and 95 (Smart) repeated data segments seen at the receiver
- Trace exported by the program, topology addresses, handshake not included

18 / 23 Smart-RUDP

SAY

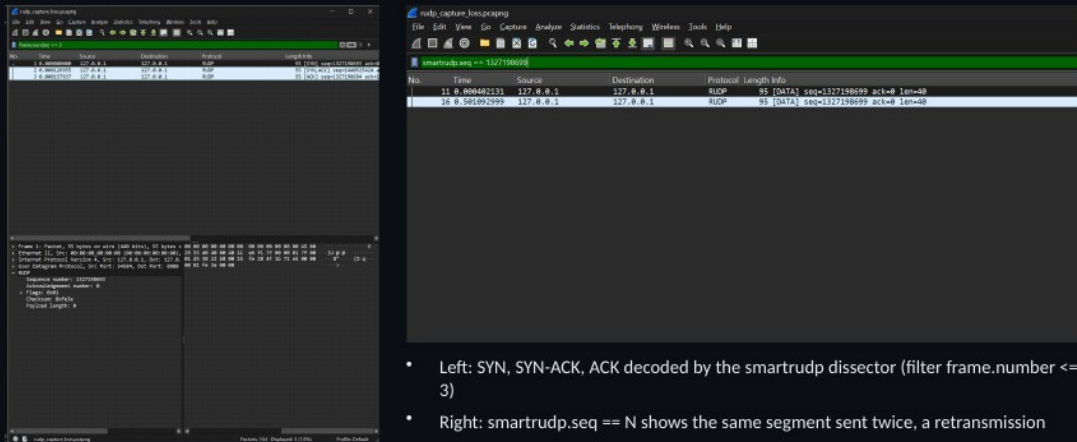
- We wrote a Lua dissector, named smartrudp, that decodes sequence, ack, flags, checksum and length.
- Useful filters: `smartrudp.flags.data == 1` and `smartrudp.seq == N`.
- In a 400-packet lossy run the receiver saw 87 (AIMD) and 95 (Smart) repeated data segments.
- The program's exported trace uses topology addresses and has no handshake.

IF ASKED Q: Why no handshake in the export? A: The export builds synthetic packets. The real tshark capture on the next slide shows the handshake.

19. Wireshark real capture

Time on this slide: 0:50

Wireshark: real capture



SAY

- Left: the first three packets, SYN, SYN-ACK and ACK, decoded by our dissector (filter frame.number <= 3).
- Right: filter smartrudp.seq == N shows the same segment twice, a retransmission.

IF ASKED If Wireshark is open live: Analyze, Decode As, UDP 8888 = RUDP, then use these two filters.

20. RUDP Chat demo

Time on this slide: 1:00

Demo: RUDP Chat over the internet

RUDP Chat • connected to 127.0.0.1:9202

you are Harsh

950 04:10 -> delivered

Harsh: test message 52/60

954 04:10 -> delivered

Harsh: test message 53/60

959 04:10 -> delivered

Harsh: test message 54/60

964 04:10 -> delivered

Harsh: test message 55/60

969 04:10 -> delivered

Harsh: test message 56/60

974 04:10 -> delivered

Harsh: test message 57/60

979 04:10 -> delivered

Harsh: test message 58/60

984 04:10 -> delivered

Harsh: test message 59/60

989 04:10 -> delivered

Harsh: test message 60/60

990 04:10 -> delivered

PACKET LOSS TO SIMULATE: 10%

450 39 388

packets sent lost resent

WINDOW CONTROLLER

Smart-RUDP (Q-learning)

in flight 0 / window 3 cwnd 3.4 queued 0
last action x1.0

Send 60 test messages

60 msgs in 1.3 s (smart)

PROTOCOL LOG

04:10:20 3 duplicate ACKs: fast retransmit from #53

04:10:35 3 duplicate ACKs: fast retransmit from #58

04:10:35 network lost message #58 (retransmission)

04:10:35 3 duplicate ACKs: fast retransmit from #58

04:10:35 network lost message #60 (retransmission)

04:10:35 3 duplicate ACKs: fast retransmit from #58

04:10:35 stress test done: 60 messages in 1.3 s (smart)

04:10:35 smart: rtt x1.0, loss 0%, cwnd -> 3.4 (x1.0)

Send

20 / 23 Smart-RUDP

- Each message is one RUDP segment: numbered, checksummed, ACKed in order
- Browser UI via a small gateway; terminal version too
- Two machines linked over Tailscale (laptop and phone)
- Loss slider: lost messages are resent and still arrive in order
- Window controller switch: fixed, AIMD or Smart-RUDP (Q-learning, learns live)
- Retrained for the chat: faster than AIMD on 5 of 6 held-out paths, equal on 1 (Table 11)
- Stress test, 48 unseen paths: table 3.73 s, deep (DQN) 3.20 s, AIMD 7.16 s
- Deep agent is seed-sensitive, so picked on validation paths

SAY

- The protocol in use: each message is one numbered, checksummed RUDP segment, ACKed in order.
- There is a terminal version and a browser version; two devices were linked over Tailscale, laptop and phone.
- Drag the loss slider: lost messages are resent and still arrive in order. The right panel shows sent, lost and resent counts and the log.
- There is a controller switch: fixed window, AIMD, or Smart-RUDP. For the chat we retrained Smart-RUDP on simulated paths: it was faster than AIMD on 5 of 6 held-out paths and equal on 1, because AIMD resets to one packet on random loss. Say it is simulated, and that the agent may not shrink the window without a congestion signal. Stress test: 48 random paths the agent never trained on, 3 seeds each. Mean delivery time was 3.73 seconds for Smart-RUDP against 7.16 for AIMD. Smart-RUDP was faster on 27 paths, slower on 5, within 5 percent on 16. So on our data, in simulation, it beats AIMD in the chat setting. There is also a deep neural-network option in the controller dropdown; it gave 3.20 seconds mean on the stress paths against 3.73 for the table.

IF ASKED Q: Is that a fair win? A: Both start from the same window of 10; held-out paths and seeds; honest limits: simulated, and without the no-shrink rule the agent lost. Q: Did it win in Section 8? A: No, that result stands. Q: Where does it lose? A: Tight bottleneck queues and very high loss; the fixed window of 8 is hard to beat there. A broader-trained agent did worse, so we kept the first.

21. More evidence: baselines, SACK, reordering

Time on this slide: 1:00

More evidence: baselines, SACK, reordering

48 unseen paths (simulated)	Mean	Median
Plain UDP	68% delivered	-
TCP model (selective ACK)	4.56 s	1.48 s
RUDP fixed window	3.88 s	2.00 s
RUDP AIMD	7.16 s	2.72 s
Smart-RUDP, table	3.73 s	1.99 s
Smart-RUDP, deep	3.20 s	1.79 s

The TCP row is a model, not kernel TCP. Real netem numbers need a Linux machine: `netem_baselines.sh`.

- Selective ACK (simulated): AIMD mean 7.16 s to 4.90 s, median 2.72 s to 1.44 s; faster on 34 of 48 paths. Go-Back-N is the main weakness.
- Reordering (20 ms jitter): AIMD 0.82 s to 6.79 s; deep agent 0.93 s; SACK removes most of the damage.
- 20-run intervals for the C experiments: Smart slower on 5G-like (+0.92 s) and satellite-like (+6.3 s), tie on lossy.

21 / 23 Smart-RUDP

SAY

- The abstract also promised TCP and plain UDP baselines, so we ran them on the same 48 unseen simulated paths.
- Plain UDP delivered only 68 percent of the messages, so reliability is needed. Our TCP model, with selective acknowledgments, has the lowest median at 1.48 seconds.
- Smart-RUDP deep is at 3.20 seconds mean and is about even with the TCP model path by path: faster on 22, slower on 23.
- The gap to TCP is mostly Go-Back-N. A selective-ack version of our transport cut AIMD from 7.16 to 4.90 seconds in simulation.
- Reordering hurts Go-Back-N badly: AIMD goes from 0.82 to 6.79 seconds at 20 milliseconds of jitter. The learned agents cope better and selective ack removes most of the damage.
- For the C experiments we repeated 20 runs per mode: Smart-RUDP is still slower on 5G and satellite and ties on lossy.

IF ASKED Q: Is the TCP row real TCP? A: No, it is a model with the same window cap; real kernel TCP needs netem on Linux, and we provide the script. Q: Why did you not implement SACK in C? A: Time; the simulation shows it is the biggest win and it is the first item of future work. Q: Is 20 runs enough? A: The intervals exclude zero for 5G and satellite and include zero for lossy.

22. Learnings and future work

Time on this slide: 0:50

Learnings and future work

Learned

Reliability is a set of simple rules working together.
Congestion control matters as much as retransmission.
Testing tools fail in surprising ways.

Next

Selective ACK in the C code (largest gain in simulation)
Deep agent in the C transport
Kernel netem and real multi-machine tests

Limits: Go-Back-N wastes bandwidth and suffers under reordering; C tests use an emulated path; the tabular agent did not beat AIMD in C.

22 / 23 Smart-RUDP

SAY

- Learned: reliability is a set of simple rules working together; congestion control matters as much as retransmission; testing tools fail in surprising ways.
- Next: Selective Repeat with SACK, a stronger learning agent, adaptive timeout, and tests on a real multi-machine network.
- Name the limits before she does: Go-Back-N, loopback-only protocol tests, Smart-RUDP not beating AIMD.

IF ASKED End on the limits and the fix order: SACK, adaptive timeout, real-network tests.

23. Thank you

Time on this slide: 0:15

Thank you

`github.com/Capsturrrr/rudp`

Harshavardhan Mallela Rohit Arya Shloak Sinha

SAY

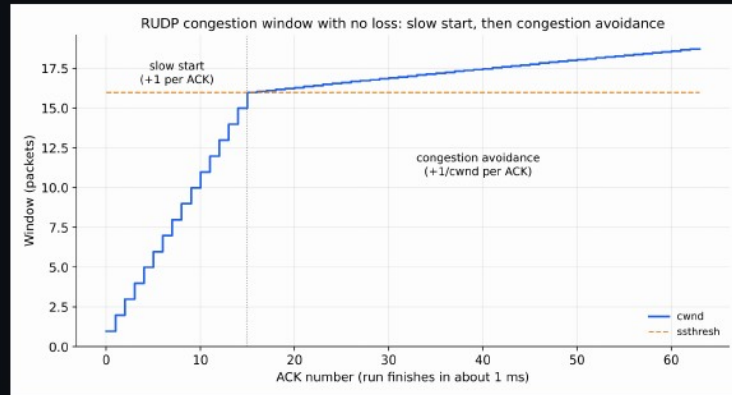
- Thank you. The code, report and captures are at github.com/Capsturrrr/rudp. Happy to take questions.

IF ASKED Pause and smile; hand over to questions.

24. Appendix: slow start

Time on this slide: -

Appendix: slow start and congestion avoidance



- cwnd starts at 1
- +1 per ACK until ssthresh = 16
- Then +1/cwnd per ACK
- 62 packets, final cwnd 18.71

SAY

- Only if asked: the window starts at 1, adds 1 per ACK until the threshold 16, then 1/window per ACK. In the 62-packet run the final window was 18.71.

Appendix Smart-RUDP

IF ASKED Use this slide to answer 'show me congestion avoidance'.

Hard questions and short answers (1 of 2)

Is this a lab exercise?

A lab sends datagrams. We built reliability and congestion control on top, measured them, and used them in a real chat and a routed network.

Why not TCP?

TCP is in the kernel and cannot be changed. Building our own makes every mechanism visible. QUIC and HTTP/3 do the same.

Did Smart-RUDP win?

Experiments: tied on one path, slower on two, and we say so. Chat, retrained on simulated paths: faster than AIMD on 5 of 6 held-out paths, equal on 1, because AIMD resets to 1 on every timeout. Deep RL: a small DQN in the chat gateway gave 3.20 s mean on the same stress paths (training is seed-sensitive; chosen on validation paths). Stress test on 48 unseen random paths: mean 3.73 s Smart vs 7.16 s AIMD; faster on 27, slower on 5, within 5% on 16. Simulated;

Why Go-Back-N?

Simpler to build and explain. It wastes bandwidth under loss; Selective Repeat with SACK is future work.

Tests ran on one machine?

Yes, with emulated delay, loss and jitter. They do not prove internet performance; the report says so. Chat between laptop and phone is a small real-network check.

Hard questions and short answers (2 of 2)

What does Packet Tracer prove?

The network design: OSPF, VLANs, ACL, 70/71 failover. The protocol is validated separately with emulation and Wireshark.

Peer disappears?

Sender keeps retransmitting on timeout. A connection abort after repeated timeouts is not built; it is the fix we would add.

Does the checksum catch errors?

16-bit one's-complement sum, like UDP/TCP. Corrupted packets fail and are retransmitted. It cannot catch every error pattern.

Why no handshake in the exported trace?

The export builds synthetic packets. The real tshark capture shows SYN, SYN-ACK, ACK.

Is Q-learning really learning?

80 states, 5 actions, 40 episodes; the learning-curve figure shows it. Small and tabular, with limited training.